

Nicht-blockierender Bewässerungs-Scheduler (ESP32 / MicroPython)

Diese Notiz fasst die Struktur für eine zeitgesteuerte Bewässerung zusammen: minutenexakte Trigger (z. B. Minute 0/20/40), 10 s Laufzeit pro Kreis, ohne Blockieren und mit sauberer Zustandsmaschine pro Kreis.

****Prinzip****

- Zeitbasis: `localtime\_ch()` (Zürich-Zeit) im 1■Hz■Takt abfragen.
- Konfiguration: Pro Kreis Menge geplanter Minuten (z. B. {0,20,40}) und Laufzeit (z. B. 10 s).
- Zustände: `IDLE` → `RUNNING` (10 s) → `COOLDOWN` (Rest der Minute) → `IDLE`.
- Zeitfenster: Nur zwischen `dayHourActive` und `dayHourInactive` schalten.

Beispielcode

```
# --- Hardwarezuordnung ---
from machine import Pin
from time import sleep, ticks_ms, ticks_diff
from time_zh import localtime_ch # liefert (t_local, _, _)
import _thread

pump1 = Pin(12, Pin.OUT) # Beispiel-Pins
pump2 = Pin(13, Pin.OUT)

# --- Konfiguration ---
WATERING_WINDOW = (7, 21) # 07:00-21:00 aktiv (lokale Zeit)
SCHEDULES = {
    "c1": {"minutes": {0, 20, 40}, "duration_s": 10, "pin": pump1},
    "c2": {"minutes": {10, 30, 50}, "duration_s": 10, "pin": pump2},
}

# --- Zustände ---
IDLE, RUNNING, COOLDOWN = 0, 1, 2

state = {
    "c1": {"mode": IDLE, "t_start_ms": 0, "last_min": None},
    "c2": {"mode": IDLE, "t_start_ms": 0, "last_min": None},
}

def in_active_window(hour_now: int) -> bool:
    start_h, stop_h = WATERING_WINDOW
    return start_h <= hour_now < stop_h if start_h < stop_h else (hour_now >= start_h or hour_now < stop_h)

def set_output(pin: Pin, on: bool):
    try:
        pin.value(1 if on else 0)
    except Exception:
        pass

def process_circuit(cid: str, hour: int, minute: int, now_ms: int):
    cfg = SCHEDULES[cid]
    st = state[cid]

    # 1) Fenster prüfen
    if not in_active_window(hour):
        if st["mode"] != IDLE:
            set_output(cfg["pin"], False)
```

```

    st["mode"] = IDLE
    st["last_min"] = minute
    return

# 2) Zustandslogik
if st["mode"] == IDLE:
    if (minute in cfg["minutes"]) and (st["last_min"] != minute):
        set_output(cfg["pin"], True)
        st["t_start_ms"] = now_ms
        st["mode"] = RUNNING
        st["last_min"] = minute

    elif st["mode"] == RUNNING:
        if ticks_diff(now_ms, st["t_start_ms"]) >= (cfg["duration_s"] * 1000):
            set_output(cfg["pin"], False)
            st["mode"] = COOLDOWN

    elif st["mode"] == COOLDOWN:
        if st["last_min"] != minute:
            st["mode"] = IDLE
            st["last_min"] = minute

def scheduler_task():
    # Initiale Minutenmarker setzen
    t_local, _, _ = localtime_ch()
    init_min = t_local[4]
    state["c1"]["last_min"] = init_min
    state["c2"]["last_min"] = init_min

    while True:
        try:
            t_local, _, _ = localtime_ch()
            hour, minute = t_local[3], t_local[4]
            now_ms = ticks_ms()

            process_circuit("c1", hour, minute, now_ms)
            process_circuit("c2", hour, minute, now_ms)

            sleep(1) # 1 Hz Takt
        except Exception:
            sleep(2)

# Thread starten (nachdem Hardware/Cloud steht)
_thread.start_new_thread(scheduler_task, ())

```

****Warum diese Struktur?***

- Exakte Minuten■Trigger: Schaltet nur einmal pro geplanter Minute (Wächter `last\_min`).
- Nicht blockierend: Laufzeitmessung mit `ticks\_ms()` statt `sleep(10)`.
- Fenster■Kontrolle: Bei Fensterende sofort OFF und zurück auf `IDLE`.
- Skalierbar: Mehr Kreise oder dynamische Pläne leicht erweiterbar.

****Temperaturabhängige Minuten■Pläne (Beispiel):***

```
``python
```

```

def minutes_for_temp(t_c):
    if t_c < 20: return {30} # 1x/h
    if t_c < 26: return {0, 30} # 2x/h
    return {0, 20, 40} # 3x/h
...

```

Dann in `process\_circuit(...)` statt `cfg["minutes"]` die Funktion verwenden.